

# JavaScript - kérdések és válaszok

Véletlen számok, megjelenítési méret, kép pozicionálása, egéresemények

## Kérdés: Véletlen szám generálása javascriptben?

JavaScriptben a `Math.random()` a legegyszerűbb megoldás. 0 (beleértve) és 1 (kizárva) közötti lebegőpontos számot ad:

```
const r = Math.random(); // pl. 0.7351...
```

## Egész szám egy tartományban (min és max is benne van)

```
function randomInt(min, max) {  
  return Math.floor(Math.random() * (max - min + 1)) + min;  
}
```

```
randomInt(1, 6); // dobókocka: 1-6
```

## Lebegőpontos szám tartományban

```
function randomFloat(min, max) {  
  return Math.random() * (max - min) + min;  
}
```

## Véletlen elem egy tömbből

```
const lista = ['alma', 'körte', 'szilva'];  
const elem = lista[Math.floor(Math.random() * lista.length)];
```

## Kriptográfiailag biztonságos véletlen szám

Jelszó, token, azonosító esetén ezt érdemes használni, mert a `Math.random()` nem biztonságos:

```
const tomb = new Uint32Array(1);  
crypto.getRandomValues(tomb);  
console.log(tomb[0]);
```

```
// UUID generálása  
const id = crypto.randomUUID();
```

A `crypto.getRandomValues()` és a `crypto.randomUUID()` a modern böngészőkben és a Node.js-ben is elérhető.

## Kérdés: Hogyan tudom lekérdezni a megjelenítés méretét javascriptben? Hogyan tudom beállítani egy image pozícióját javascriptben?

## Megjelenítési méret lekérdezése

### A böngészőablak látható területe (viewport):

```
const szelesseg = window.innerWidth;  
const magassag = window.innerHeight;
```

### Görgetősáv nélkül (a dokumentum látható területe):

```
const szelesseg = document.documentElement.clientWidth;  
const magassag = document.documentElement.clientHeight;
```

## A teljes képernyő mérete (monitor):

```
const kepernyoSz = screen.width;  
const kepernyoM = screen.height;
```

## Átméretezés figyelése:

```
window.addEventListener('resize', () => {  
  console.log(window.innerWidth, window.innerHeight);  
});
```

## Kép pozíciójának beállítása

A pozicionáláshoz a képnek **position: absolute** vagy **fixed** stílus kell, utána a **left** és **top** értékekkel mozgatható:

```
const kep = document.getElementById('kep');  
  
kep.style.position = 'absolute';  
kep.style.left = '100px';  
kep.style.top = '50px';
```

## Példa: kép középre helyezése az ablakban:

```
const kep = document.getElementById('kep');  
kep.style.position = 'absolute';  
  
function kozepre() {  
  kep.style.left = (window.innerWidth - kep.offsetWidth) / 2 + 'px';  
  kep.style.top = (window.innerHeight - kep.offsetHeight) / 2 + 'px';  
}  
  
kep.onload = kozepre; // a kép betöltése után ismert a mérete  
window.addEventListener('resize', kozepre);
```

**Alternatíva:** **transform**, ami animációnál simább, mert nem váltja ki az oldal újratördelését:

```
kep.style.transform = `translate(${x}px, ${y}px)`;
```

## Az aktuális pozíció lekérdezése:

```
const rect = kep.getBoundingClientRect();  
console.log(rect.left, rect.top, rect.width, rect.height);
```

A két témát kombinálva pl. véletlen helyre is teheted a képet:

```
kep.style.left = Math.random() * (window.innerWidth - kep.offsetWidth) + 'px';  
kep.style.top = Math.random() * (window.innerHeight - kep.offsetHeight) + 'px';
```

## Kérdés: Van olyan javascript eseménykezelő, amivel azt lehet érzékelni, hogy egy adott objektum fölé kerül a kurzor?

Igen, erre a **mouseenter** és a **mouseover** esemény való, a párjuk pedig a **mouseleave** és a **mouseout**, amelyek a kurzor távozását jelzik.

```
const kep = document.getElementById('kep');  
  
kep.addEventListener('mouseenter', () => {  
  console.log('A kurzor a kép fölé került');  
});  
  
kep.addEventListener('mouseleave', () => {  
  console.log('A kurzor elhagyta a képet');  
});
```

## A két páros közötti különbség

- A `mouseenter` / `mouseleave` csak egyszer fut le, amikor a kurzor belép az elembe vagy kilép belőle. A gyerekelemek között mozogva nem aktiválódik újra, és nem buborékol. Általában ez a kényelmesebb választás.
- A `mouseover` / `mouseout` buborékol, és akkor is lefut, amikor a kurzor az elem egy gyerekelemére lép. Ez akkor hasznos, ha egy szülőelemen akarod figyelni sok gyerekelem eseményét (event delegation).

## Érintőképernyőt is kezelő változat

A `pointerenter` / `pointerleave` ugyanígy működik, de egér, toll és érintés esetén is:

```
kep.addEventListener('pointerenter', () => { /* ... */ });
```

## Példa: a kép elugrik egy véletlen helyre, amikor fölé viszed az egeret

```
kep.style.position = 'absolute';
```

```
kep.addEventListener('mouseenter', () => {  
  kep.style.left = Math.random() * (window.innerWidth - kep.offsetWidth) + 'px';  
  kep.style.top = Math.random() * (window.innerHeight - kep.offsetHeight) + 'px';  
});
```

Ha csak vizuális hatás kell, például kiemelés vagy nagyítás, JavaScript nélkül, CSS-sel is megoldható:

```
#kep:hover {  
  transform: scale(1.1);  
}
```